



Ollama

-- Build AI locally

By **Prudvi**





Proprietary vs Open Source Models

Aspect	Proprietary Models	Open Source Models
Models	GPT-4, Claude, Gemini etc.	Llama, Mistral, DeepSeek, Qwen etc.
Performance	State-of-the-art, highest accuracy and reasoning	Good and improving, but may be slightly lower than top proprietary models
Setup	No setup required (ready via API/UI)	Requires installation and environment setup
Cost	Paid (API/token-based pricing)	Mostly free (only compute cost)
Pricing	Pricing may increase over time	No payment barrier
Data Privacy	Data sent to external servers	Data stays local (full privacy)
Data Sovereignty	Limited control over data location/compliance	Full control, supports strict regulations
Internet Dependency	Requires internet	Can work fully offline
Hardware Requirement	No local hardware needed	Requires CPU/GPU resources
Optimization Effort	Managed by provider	Needs manual optimization (quantization, formats)
Customization	Limited customization, Vendor has to support	Highly customizable and fine-tunable



Direct Open source Vs Ollama

Category	Direct Open Source (e.g., Hugging face)	Ollama
Setup Complexity	High – manual install, dependencies, configs	Low – simple install & run
Model Management	Manual – download, convert, organize models	Automatic – pull/run models easily
Optimization	Hard – manual quantization & tuning required	Built-in – optimized out-of-the-box
Beginner Friendly	No – requires technical knowledge	Yes – very easy to start
Production Readiness	Moderate – needs custom setup & scaling	High – structured runtime, APIs



Setup & Requirements

Hardware Requirements :

- CPU can run models but GPU gives **massive speed boost (5x–20x)**
- Ollama **automatically uses GPU** if available

Model Size	RAM	VRAM (GPU)	CPU Requirement	GPU Requirement
7B	8 GB (min)16 GB (recommended)	6–8 GB	4–6 cores (i5 / Ryzen 5)	Entry GPU (RTX 3060)
13B	16–32 GB	12–16 GB	8–12 cores (i7 / Ryzen 7)	Mid-range GPU
30B	32–64 GB	20–24 GB	12–16 cores	High-end GPU (RTX 3090 / A100)
70B	64 GB+	40 GB+	16+ cores (Xeon / EPYC)	Multi-GPU / Server-grade

Software requirements:

- **OS:** Windows, macOS and Linux
- Install Ollama <https://ollama.com/download>

For GPU:

- NVIDIA - NVIDIA Drivers + CUDA Toolkit - Full ecosystem support
- AMD - Uses **ROCm** (AMD alternative to CUDA) – only Linux - Not well supported in Ollama
- Apple (M1/M2/M3) – No CUDA - Uses **Metal (Apple GPU framework)**



Ollama Usage modes

Method	What It Is	Best For	Commands / Usage
CLI (Command line Interface)	Run commands in terminal	Quick testing & chatting	ollama run llama3 ollama pull <model> ollama list ollama serve
REST API	Local HTTP server interface	App integration	POST /api/generate POST /api/chat GET /api/tags
Python Library	Python SDK for Ollama	Automation & backend logic	ollama.chat() ollama.generate() ollama.pull() stream=True
LangChain	Integration with LangChain framework	RAG & AI agents	ChatOllama(model=...) OllamaEmbeddings()



Ollama using CLI

Category	Command / Syntax	Description
Version Check	<code>ollama --version</code>	Check installed Ollama version
Run / Pull Model	<code>ollama run qwen3.5:2b</code>	Runs the model; downloads if not available locally
List Models	<code>ollama list</code> or <code>ollama ls</code>	Shows all locally available models
Start Chat	<code>ollama run qwen3.5:2b</code>	Starts interactive chat session (>>>)
Exit Chat	Ctrl + D or <code>/bye</code>	Exit the chat session
Show Commands	<code>/show</code>	Displays available in-chat commands
Model Info	<code>/show info</code>	Displays model details
Show Parameters	<code>/show parameters</code>	Displays current model parameters
Set Parameters	<code>/set parameter top_p 0.99</code>	Modify parameters like <code>top_p</code> , temperature, etc.
Documentation	https://docs.ollama.com/cli	Refer for all commands and advanced usage



Ollama using python Library

Install the python library : Pip install ollama

Import the library in code : Import ollama

Library Usage examples:

```
result = ollama.generate(model="qwen3.5:2b",prompt="explain the transformer architecture")
result = ollama.generate(model="qwen3.5:2b",prompt="explain the transformer architecture", stream=True)
result = ollama.generate(model="qwen3.5:2b",prompt="Relation between India and USA",
system="You are an funny assistant , you explain things in political direction")
```

```
result = ollama.generate(model="llama3.2:1b",
    prompt="Why python is highly used programming language?",
    system="You are an assistant who explains things in a very simple way",
    options={
        "temperature":0.3,
        "top_p":0.5,
        "top_k":45
    })
```

```
local_models= ollama.list() # to list the available models in local
```

To pull the model

```
model_name="llama3.2:1b"
progress = ollama.pull(model_name, stream=True)
```



Ollama using Rest API

Ollama internally works using REST API hosted locally at port 11434 <http://localhost:11434/api>

Documentation : <https://docs.ollama.com/api/introduction>

Core interface that handles all model requests (/api/generate, /api/chat, etc.)

CLI and Python library are just the wrappers around the API end points

When you run:

```
ollama run llama3
```

CLI internally makes a request like:

```
POST http://localhost:11434/api/chat
```

Because of this architecture:

You can integrate Ollama with:

- Any backend (Java, Node.js, Go, etc.)
- Frontend apps

No dependency on Python or CLI — just HTTP

CLI / Python / Any App



REST API (localhost:11434)



Ollama Engine



Model (GGUF via llama.cpp)



Ollama using Langchain

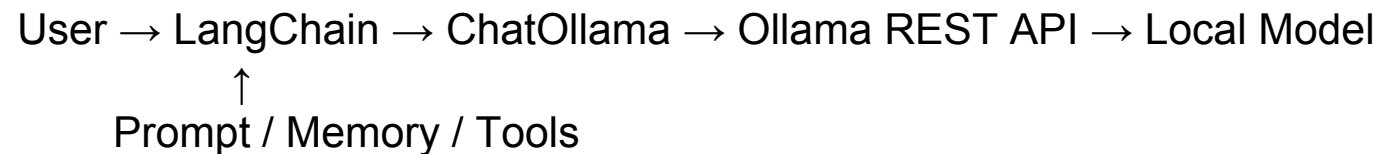
- **Ollama** → runs the model locally (via REST API at <http://localhost:11434>)
- **LangChain** → acts as the orchestration layer (chains, memory, agents)
- LangChain simply calls Ollama's API under the hood.
- Install the langchain modules to use Ollama
 - `pip install langchain langchain-community langchain-ollama`
 - `from langchain_ollama import ChatOllama`

Make sure Ollama is running by running - `ollama serve`

Pull a model:

```
ollama pull llama3
```

If “ollama serve” is not run explicitly Ollama auto start the model service





Model File

A **Modelfile** in Ollama is a configuration file used to **customize an LLM's behavior, personality, and settings**.

- The **actual intelligence (model weights)** comes from base models like Llama
- The **Modelfile controls behavior and usage**

Create a model using instructions from <https://docs.ollama.com/modelfile>

Follow the instruction form above link to create the ModelFile

To create the “Modelfile” use below table (make sure no extension to the file name)

- ollama create mymodel -f Modelfile & ollama run choose-a-model-name

Instruction	Description
<u>FROM</u> (required)	Defines the base model to use.
<u>PARAMETER</u>	Sets the parameters for how Ollama will run the model.
<u>TEMPLATE</u>	The full prompt template to be sent to the model.
<u>SYSTEM</u>	Specifies the system message that will be set in the template.
<u>ADAPTER</u>	Defines the (Q)LoRA adapters to apply to the model.
<u>LICENSE</u>	Specifies the legal license.
<u>MESSAGE</u>	Specify message history.
<u>REQUIRES</u>	Specify the minimum version of Ollama required by the model.

A large, abstract graphic consisting of several concentric, overlapping rings in shades of blue, green, and purple, framing the central text.

Thank you